Software Guidebook

Doel

Dit document bundelt de spelregels, systems en ontwerpprincipes van de game op één plek.

Gebruik dit bestand als referentie voor:

- levelprogressie;
- powerups en shopgedrag;
- boss-entrances en speciale arena's;
- UI- en feedbackprincipes.

SGB Conventie

Elk principe en elke structurele beslissing in dit document bevat expliciet een `Waarom`.

- Zonder rationale zijn regels lastig te toetsen en verouderen ze sneller.
- Met rationale kunnen we wijzigingen evalueren op intentie, niet alleen op letterlijke tekst.

Exception Visibility

Exceptions worden niet stil ingeslikt.

Waarom:

- Een fout zonder logregel is in praktijk nauwelijks te debuggen, zeker in de web-runtime.
- Stil ingeslikte exceptions maken regressies onzichtbaar en laten kapot gedrag doorgaan alsof het "gewoon niet ondersteund" is.
- Browser-, audio- en assetproblemen moeten direct herleidbaar zijn naar de callsite en het bestand dat faalt.
- Geen `except Exception: pass` in projectcode.
- Als een exception bewust wordt opgevangen om runtime of tooling overeind te houden, log dan minimaal context, exceptiontype en melding naar console of stderr.
- Bij herhaalbare runtimefouten mag logging gededupliceerd worden om spam te beperken, maar de eerste fout moet zichtbaar blijven.

Platform Parity (DEV/PROD)

Lokaal draaien en web-deploy moeten dezelfde game opleveren qua gedrag.

Waarom:

- We willen lokaal snel itereren in Python-modus, zonder dat gedrag later op web functioneel afwijkt.
- Dat voorkomt verrassingen bij release: wat lokaal speelbaar en correct is, moet online dezelfde mechanics tonen.
- Performance kan tussen runtime-omgevingen verschillen; daarom testen we naast lokaal ook regelmatig de web-build zelf.
- Geen platform-specifieke gameplayregels: obstaclelogica, powerups, hitboxes, timing en scoreprogressie zijn identiek op desktop en web.
- Geen content-splitsing per platform: levels, patterns en balanswaarden worden centraal beheerd en niet gedupliceerd in `if IS_WEB` versus lokaal.
- Platform-afhankelijke code mag alleen voor technische runtime-zaken (bijv. input/audio unlock, packaging, pad-resolutie), nooit voor gamebalans of mechanics.
- Performanceproblemen worden opgelost met generieke optimalisaties die overal gelden (assets, renderpad, objectbeheer), niet met afwijkende spelregels op web.

Framework Boundary

De map `processing/` is frameworkcode en heeft een andere wijzigingsdrempel dan gamecode zoals `dino_game.py`.

Waarom:

- Een wijziging in `processing/` heeft een veel grotere blast radius dan een wijziging in één sketch.
- Frameworkwijzigingen veranderen impliciet het contract voor alle sketches en moeten daarom als library-werk behandeld worden, niet als lokale bugfix.
- Als een bug alleen in één game of sketch zichtbaar is, lossen we die standaard eerst in de appcode op. Alleen bij bewezen frameworkschuld en expliciete afstemming passen we `processing/` aan.
- Geen ad-hoc wijzigingen in `processing/` om een bug in één sketch te fixen.
- Eerst het owning codepad in de app zelf onderzoeken.
- Als een frameworkaanpassing toch nodig lijkt, eerst expliciet afstemmen en idealiter apart behandelen als library-bug of library-change.

Web Runtime en Packaging

De webbuild bestaat niet uit "los wat Python-bestanden in de browser draaien", maar uit een kleine bootstraplaag plus een gecomprimeerde app-bundle.

Waarom:

- Zonder dit mentale model is het lastig te begrijpen waarom lokaal en productie kunnen verschillen terwijl de broncode zelf gelijk lijkt.
- De browser voert de app niet rechtstreeks uit de repository uit, maar uit een opgebouwde en verpakte webbundle.
- Voor debugging van webdeploys moet duidelijk zijn welk deel runtime is, welk deel appcode is, en welk deel uit lokale mirror of externe CDN komt.

Wat is `stage.tar.gz` ?

- Tijdens de build wordt eerst een tijdelijke stage-map opgebouwd in `.web-build/stage/` .
- Daarin worden de app-entrypoint (`main.py`), `processing/` , `assets/` en enkele begeleidende bestanden gekopieerd.
- Pygbag verpakt die stage-map vervolgens als web-app-output, waaronder `index.html` en `stage.tar.gz` .
- In een normale browserdeploy wordt vooral `stage.tar.gz` gebruikt als app-bundle; die wordt door de bootstrapcode opgehaald en uitgepakt in de virtuele filesystem van de wasm-runtime.
- De Python-code van de game zit dus inderdaad in `stage.tar.gz` , samen met assets en frameworkbestanden.
- De CPython WebAssembly-runtime zelf zit daar niet in; die komt uit de pygbag-runtimebestanden zoals `pythons.js` , `main.js` en `main.wasm` .

Waarom bestaat die tarball überhaupt?

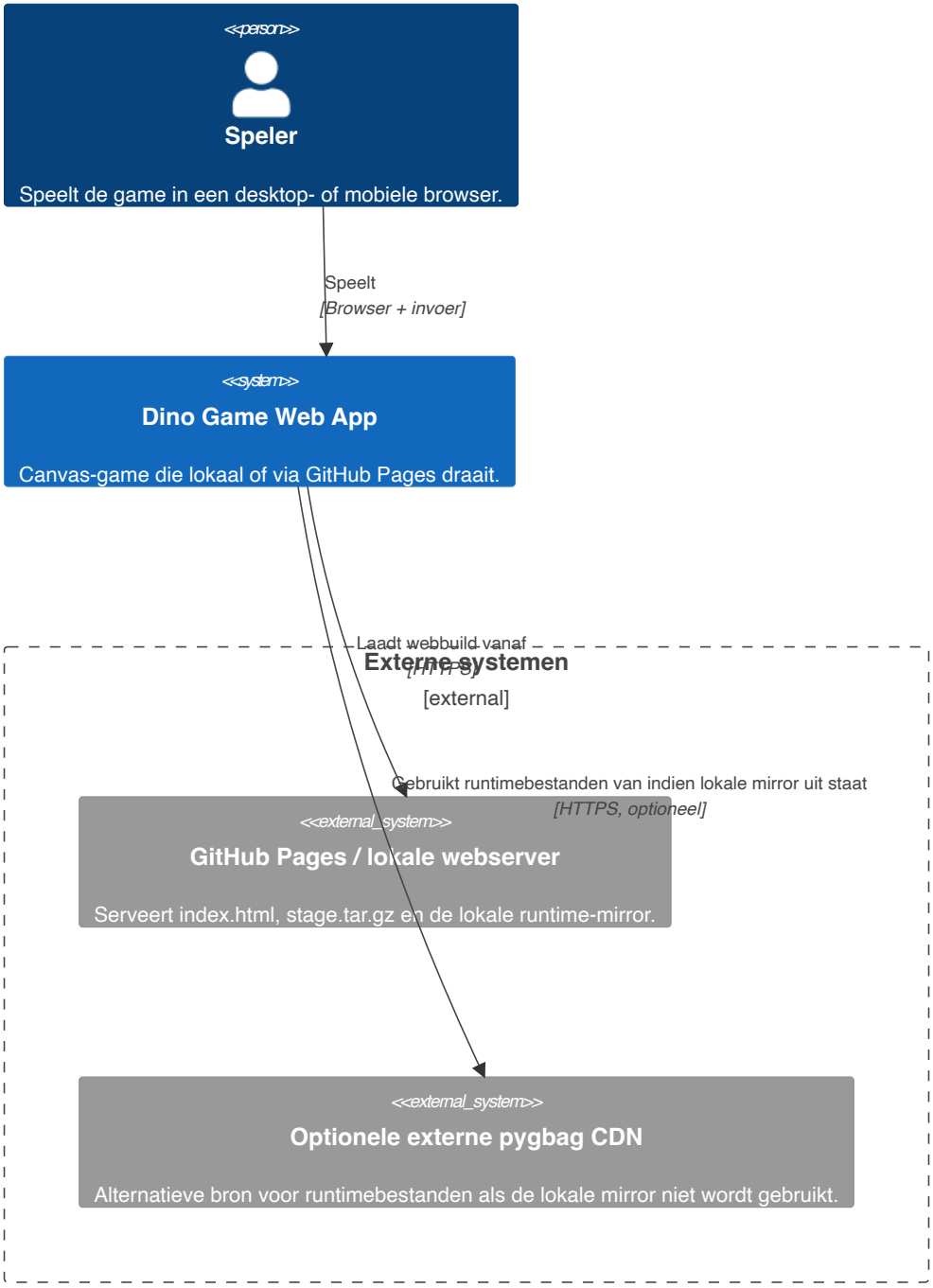
- De browser kan niet direct een hele Python-projectmap als lokale directory mounten.
- Een gecomprimeerde bundle maakt het mogelijk om de volledige app als één payload te downloaden en daarna in de virtuele wasm-filesystem uit te pakken.
- Daardoor kan `assets/main.py` in de browser draaien alsof het een gewone projectmap is.
- Het verschil tussen lokale preview en productie zit daardoor vaak niet in "de repo", maar in welke `stage.tar.gz` daadwerkelijk wordt geserveerd.

Rol van CDN versus lokale mirror

- De appcode en game-assets horen uit de eigen build-output te komen, dus uit `stage.tar.gz` en lokale bestanden onder `.web-build/output/` .
- De pygbag-runtimebestanden kunnen uit een externe CDN of uit een lokale mirror `cdn/0.9.3/` komen.
- In dit project staat `LOCAL_CDN` standaard op `1` in de buildscriptconfiguratie en de GitHub Pages workflow zet `LOCAL_CDN=1` ook expliciet aan.
- "CDN" is hier dus niet bedoeld als uitzondering, maar als lokaal meegesynchroniseerde runtime-map binnen de eigen deploy-output.
- Een verschil tussen lokaal en productie wijst in dit model meestal op een verschil in de daadwerkelijk geserveerde bundle of runtimebestanden, niet per se op een verschil in repository-broncode.

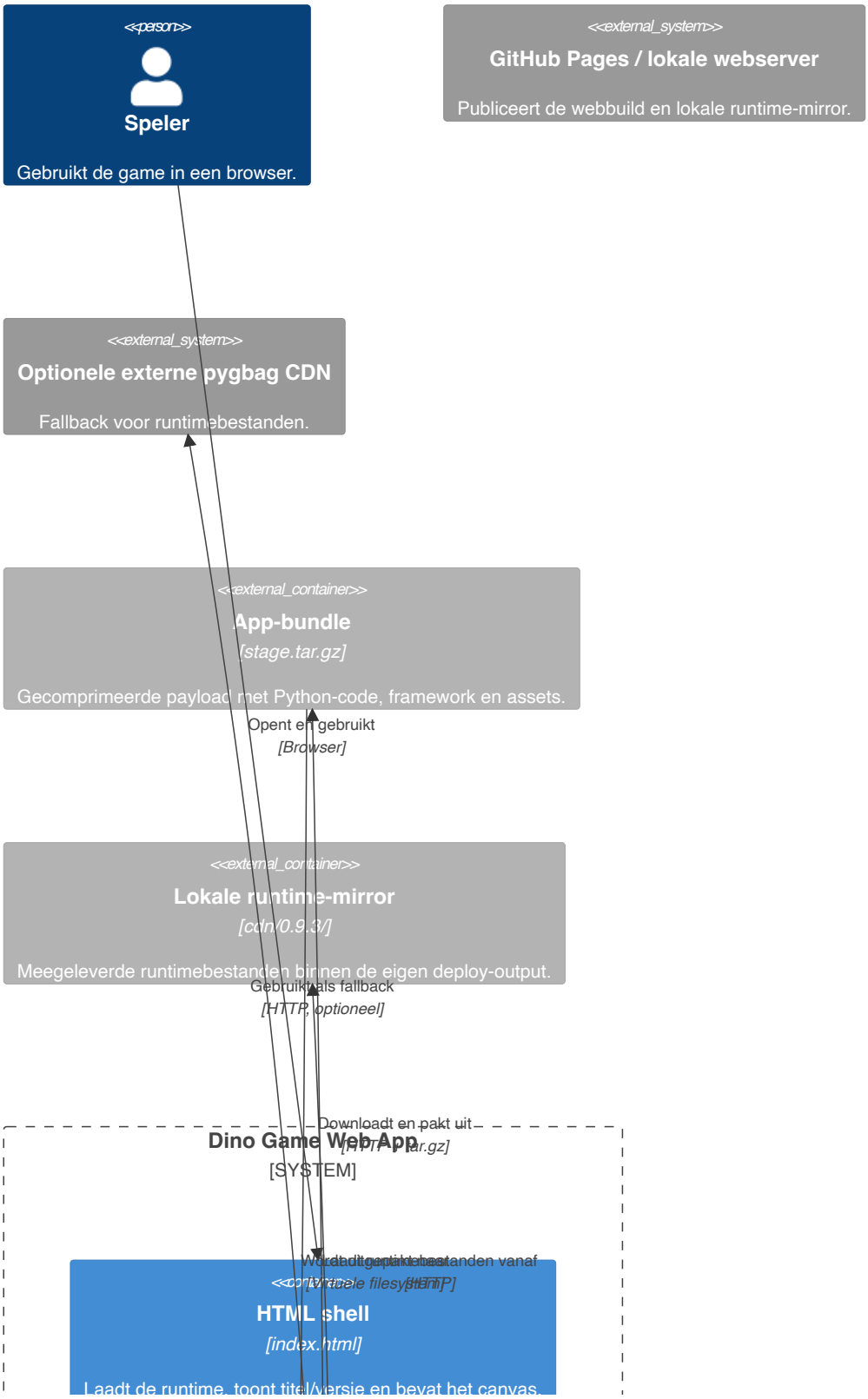
C4 Context

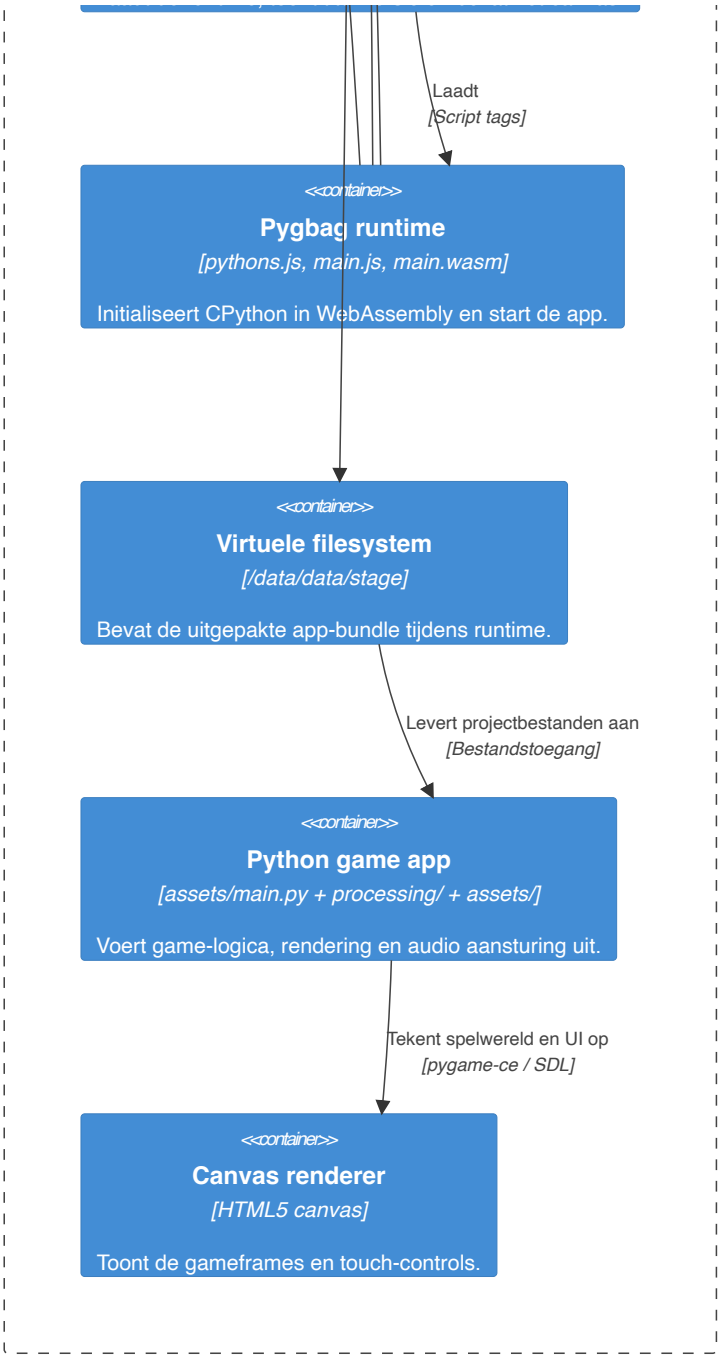
Dino Game - System Context



C4 Container

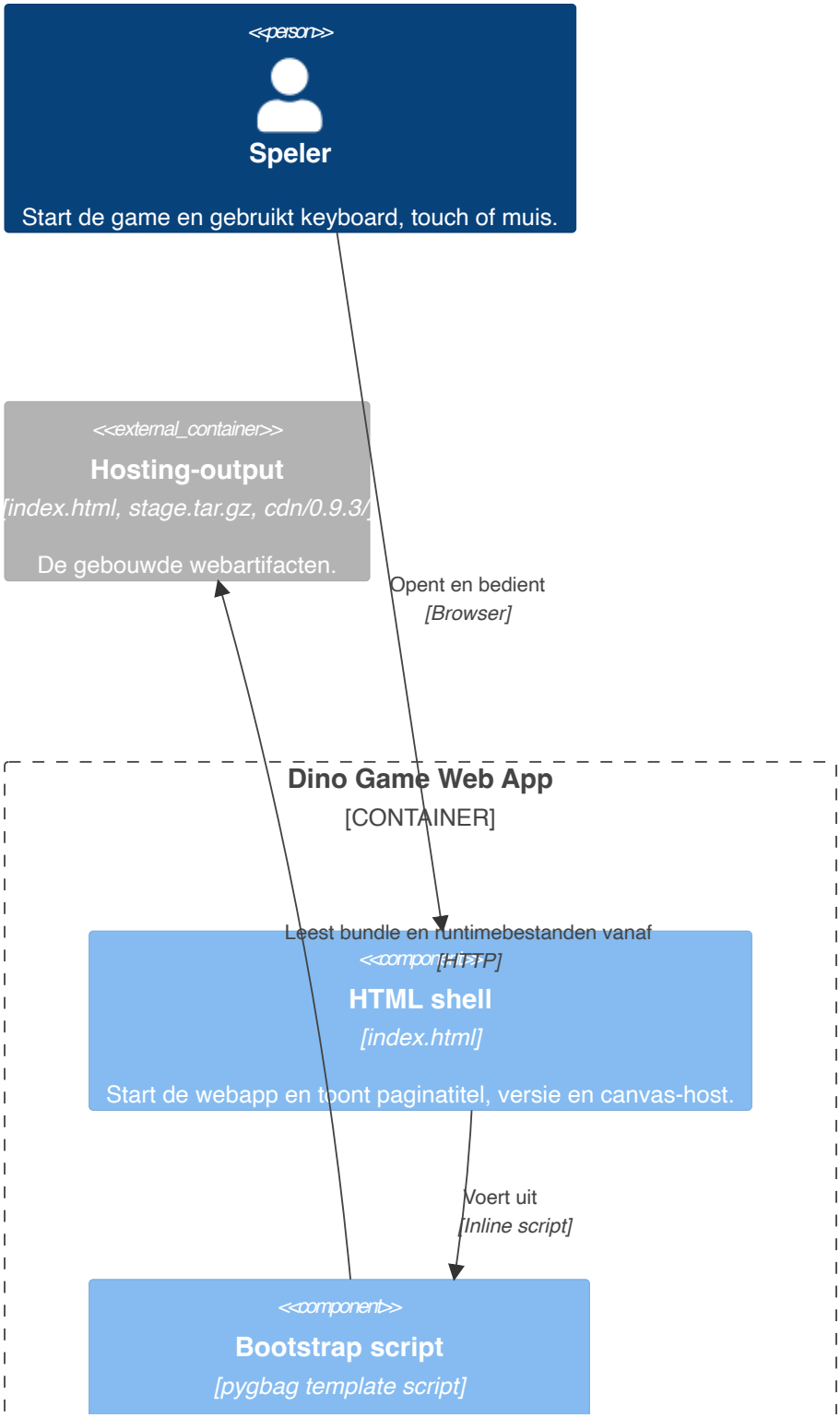
Dino Game - Container Diagram

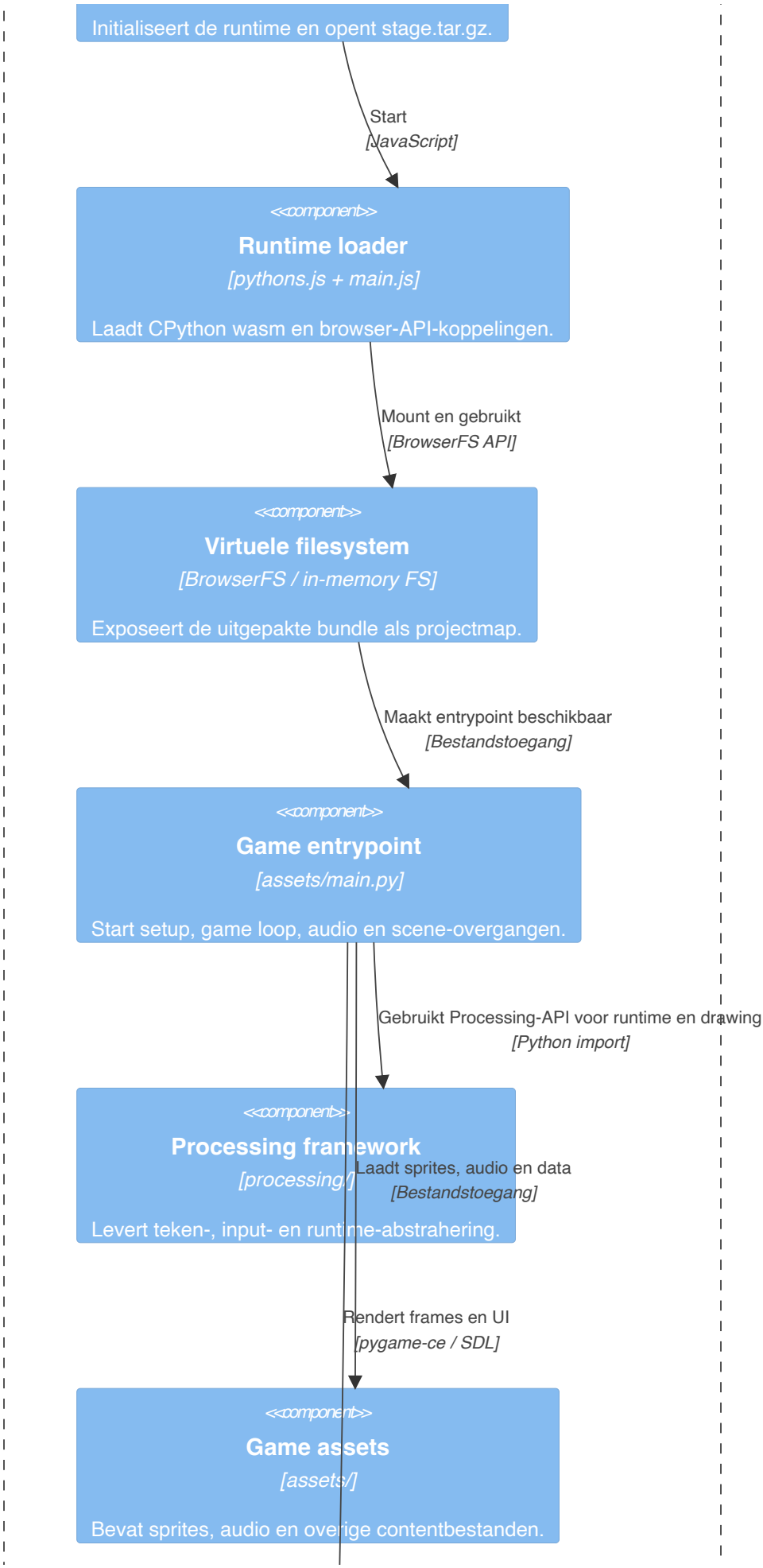


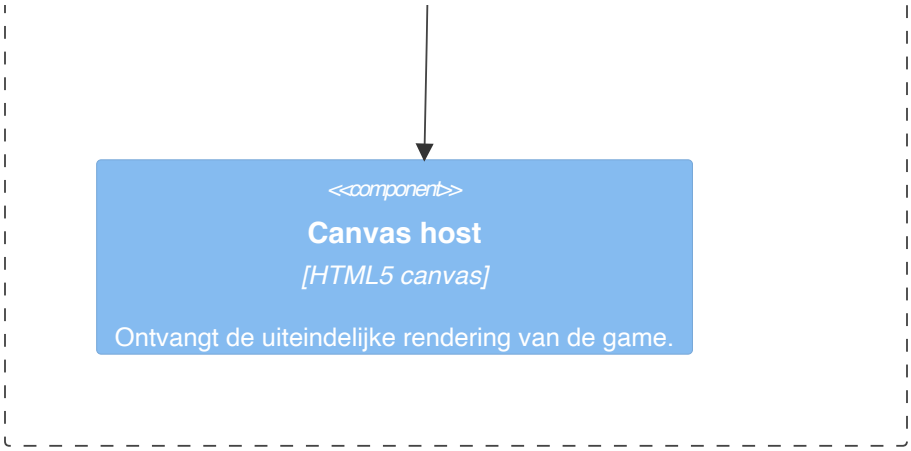


C4 Component

Dino Game - Component Diagram of the Web Runtime




Initialiseert de runtime en opent stage.tar.gz.
Start
[JavaScript]
<<component>>
Runtime loader
[pythons.js + main.js]
Laadt CPython wasm en browser-API-koppelingen.
Mount en gebruikt
[BrowserFS API]
<<component>>
Virtuele filesystem
[BrowserFS / in-memory FS]
Exposeert de uitgepakte bundle als projectmap.
Maakt entrypoint beschikbaar
[Bestandstoegang]
<<component>>
Game entrypoint
[assets/main.py]
Start setup, game loop, audio en scene-overgangen.
Gebruikt Processing-API voor runtime en drawing
[Python import]
<<component>>
Processing framework
[processing/]
Levert teken-, input- en runtime-abstrahering.
Laadt sprites, audio en data
[Bestandstoegang]
Rendert frames en UI
[pygame-ce / SDL]
<<component>>
Game assets
[assets/]
Bevat sprites, audio en overige contentbestanden.



Debug-implicaties

- Een verschil tussen lokaal en productie kan ontstaan als `index.html` of `stage.tar.gz` op productie nog van een oudere build komt.
- Runtime-fouten over missende assets of onverwachte paden betekenen in dat geval meestal dat de geladen webbundle niet overeenkomt met de verwachte build-inhoud.
- Het is daarom niet genoeg om alleen de repo-bron te inspecteren; voor webbugs moet ook de daadwerkelijk geserveerde `stage.tar.gz` gecontroleerd kunnen worden.

Creative Integrity

De game gebruikt dezelfde visuele taal op meerdere plekken.

- Sprite-based visualisation (`.png`) heeft de voorkeur boven ad-hoc getekende placeholders zodra een asset beschikbaar is.
- Interactieve keuzes gebruiken kleine, duidelijke kaders met een subtiele pulse of glow.
- Shop-items mogen dus niet ineens als totaal andere UI-componenten verschijnen als ze al als iconen in HUD of shop bestaan.
- Informatie met gameplay-impact moet leesbaar zijn in de wereld zelf, niet alleen als tekst erboven.
- Gevaar krijgt altijd een visuele cue: bijvoorbeeld een landingsgloed onder een vallende bom, of een arena die oplicht bij explosies.
- Level-flow is leerbaar: obstaclevolgordes zijn in basis scripted/hardcoded per level, zodat spelers patronen kunnen herkennen en verbeteren.
- Variatie mag alleen gecontroleerd: vanaf hogere levels uitsluitend via een kleine, vooraf ontworpen set veilige patroonblokken.

- Onmogelijke combinaties zijn verboden: runtime mag geen obstacleketens genereren die niet haalbaar zijn met normale timing/spronghoogte wanneer een powerup net is afgelopen.
- Luchtlevels gebruiken decoratieve parallax-wolken als sfeerlaag en leesbare snelheidsreferentie, nooit als obstacle of misleidende hitbox.
- Menu-tekst boven de luchtlaag gebruikt karakter-afhankelijke contrastkleuren en mag nooit visueel over elkaar heen vallen; titel, startprompt en character-select copy blijven gescheiden blokken.
- Een echte sprite-asset vervangt procedurale placeholder-tekeningen zodra zo'n asset beschikbaar en geschikt is.

Semi-Realistische Continuiteit

Interacties mogen gestileerd zijn, maar de wereld mag niet abrupt vergeten wat er net nog zichtbaar of fysiek aanwezig was.

Waarom:

- De game is arcadeachtig, maar voelt sterker als objecten en gevolgen logisch doorlopen in beeld.
- Een vliegtuigcrash die direct omslaat naar een grond-lijk zonder wrak of luchtobstakels breekt de ruimtelijke continuiteit.
- Semi-realistische consequentie maakt verlies, impact en level-geometrie leesbaarder zonder dat de game een simulator hoeft te worden.
- Bestaande wereldobjecten verdwijnen niet zomaar op het moment van impact als daar geen zichtbare reden voor is.
- Een crash in `flight_mode` hoort visueel eerst als vliegtuigcrash leesbaar te blijven voordat de game reset of naar een andere state springt.
- Obstakels en arena-elementen blijven in principe zichtbaar tijdens een defeat-state als zij direct onderdeel waren van de aanleiding van die defeat.
- Stylization is toegestaan, maar alleen zolang oorzaak en gevolg voor de speler visueel navolgbaar blijven.

Character Asset Contract

Voor character poses is een runtime sprite sheet niet verplicht, maar de assetset moet zich wel gedragen alsof de frames uit één sprite sheet komen.

Begrippen:

- `canvas` : de volledige rechthoek van een spritebestand, inclusief transparante marge.
- `crop` : hoe strak het zichtbare figuur uit een groter beeld is uitgesneden.
- `grondlijn` : de denkbeeldige horizontale lijn waarop voeten, poten of wielen de grond raken.
- `anchor` : het vaste referentiepunt waarmee een pose op dezelfde plek wordt getekend als de vorige pose.

Waarom:

- Het grootste risico bij losse PNG's is niet het ontbreken van een sheet, maar inconsistente canvasmaat, baseline en anchor tussen poses.
- Een crouch-pose die strakker of anders gecropt is dan de normale pose veroorzaakt zichtbaar verspringen tijdens posewissels, ook als de hitbox zelf correct blijft.
- Een sprite sheet maakt zulke afwijkingen snel zichtbaar, maar dezelfde discipline is ook mogelijk met losse bestanden.
- Losse PNG's per state zijn toegestaan; een sprite sheet is dus geen vereiste.
- Binnen één character-set moeten alle poses exact dezelfde canvasbreedte en canvashoogte hebben: dus echt dezelfde pixelmaat per bestand binnen die set.
- Binnen één character-set delen alle poses dezelfde grondlijn: voeten of laagste contactpunt landen op dezelfde horizontale lijn.
- Binnen één character-set delen alle poses dezelfde horizontale anchor, zodat een posewissel niet naar links of rechts "springt".
- Strakke crops zijn ondergeschikt aan consistentie: voeg liever transparante marge toe dan dat één pose kleiner of verschoven binnenkomt.
- De concrete pixelmaat hoeft niet vooraf globaal voor alle characters gelijk te zijn, maar wordt per character-set expliciet gekozen zodra de eerste definitieve asset van die set wordt ingevoerd.
- Zodra zo'n maat voor een character-set is gekozen, worden nieuwe poses in die set daarop gepad of uitgelijnd in plaats van opnieuw vrij gecropt.

Foutvoorbeelden:

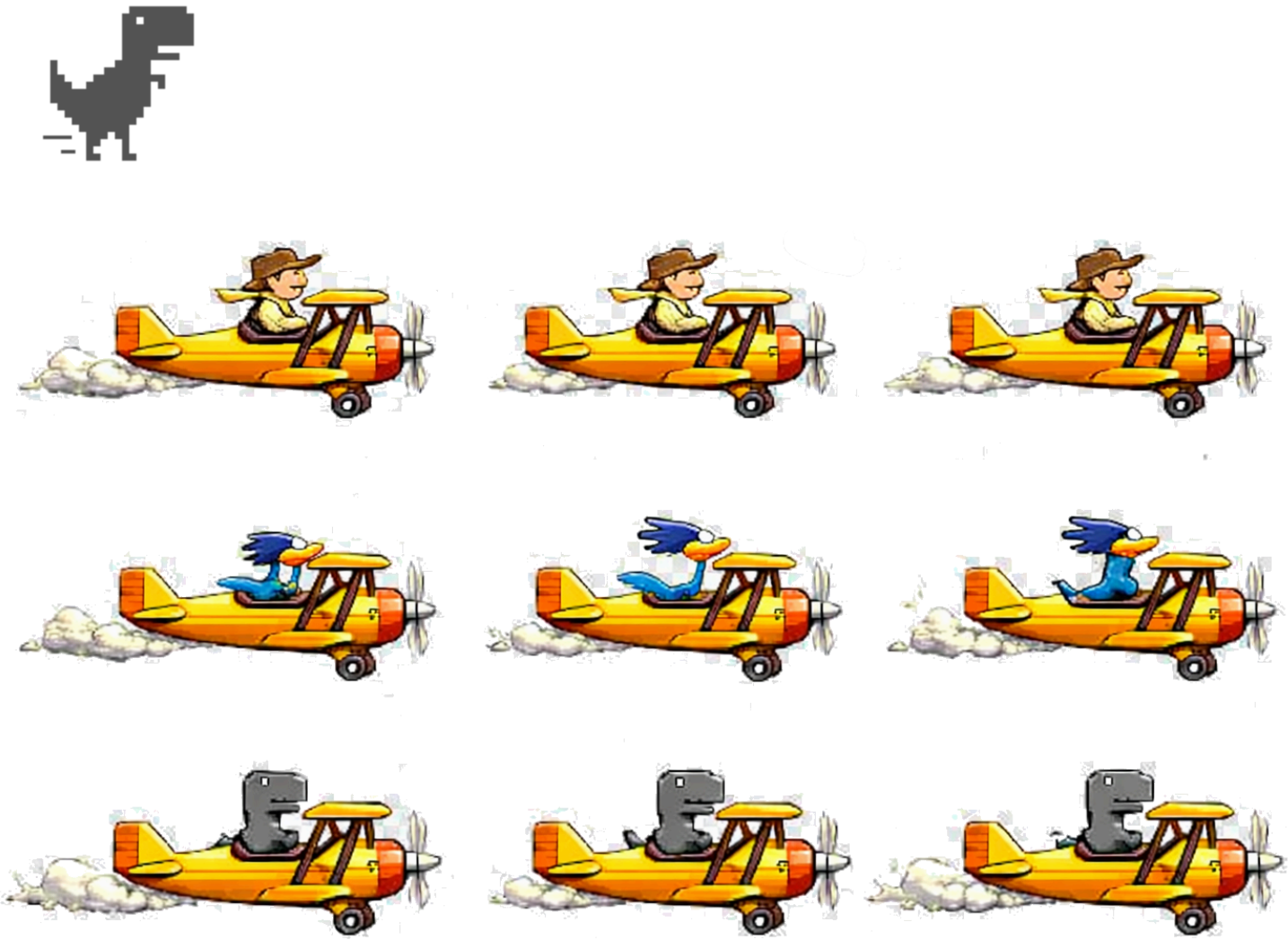
- `normaal` is 220 x 160 pixels, maar `duck` is 184 x 109 pixels. Resultaat: de sprite lijkt kleiner te worden en verschuift bij het bukken.
- `normaal` heeft 18 pixels transparante ruimte onder de voeten, maar `duck` maar 2 pixels. Resultaat: de crouch-variant zakt visueel door de grond of schiet omhoog.
- `normaal` is gecentreerd op het midden van het canvas, maar `duck` is verder naar links gecropt. Resultaat: de pose "teleporteert" horizontaal tijdens het wisselen.
- `oops` is veel strakker uitgesneden dan `normaal` . Resultaat: dezelfde character voelt ineens alsof die van schaal verandert.

Goed voorbeeld:

- Als de dino-set eenmaal op een vaste pixelmaat is gezet, moeten `normaal`, `duck`, `oops` en eventuele extra poses allemaal precies diezelfde canvasmaat, grondlijn en anchor gebruiken, ook als de zichtbare dino in de crouch-versie lager of compacter is.

Repo-voorbeelden:

- Losse sprite: `assets/dino-transparant.png`
- Sprite sheet: `assets/plane-sprite.png`



Waarom:

- De Chrome Dino-vluchtstukken voelen sterker als luchtspace wanneer de achtergrond subtiel meebeweegt.
- Parallax helpt snelheid en diepte lezen zonder extra gameplay-ruis toe te voegen.
- Wolken mogen de obstacleleesbaarheid niet aantasten; daarom blijven ze puur decoratief.
- Ook in het menu moet tekst boven de lucht onmiddellijk leesbaar blijven; gekozen karakter en luchtkleur bepalen dus mee welke donkere contrastkleur daar werkt.

Geen Defensieve Gameplay-Fallbacks

Gameplaycode mag niet stilletjes ander gedrag kiezen als een asset, API, state of aanname ontbreekt.

Waarom:

- Fallbacks maken fouten onzichtbaar en daardoor lastig te reproduceren.
- Testen wordt onbetrouwbaar als code onder onbekende omstandigheden "iets anders" doet dan het ontworpen gedrag.
- Bij AI-assisted development moeten ontbrekende assets, ontbrekende API's en kapotte aannames expliciet zichtbaar worden, anders bouwt de agent verder op een verborgen fout.
- Een hard failure met een duidelijke oorzaak is beter dan een speelbare maar verkeerde toestand.
- Geen stille fallback van ontbrekende gameplay-assets naar oude sprites.
- Geen alternatieve mechanics bij ontbrekende functies of ongeldige state.
- Geen brede `try/except` rond gameplaylogica om fouten te maskeren.
- Wel toegestaan: expliciete platform-adapters voor technische runtimeverschillen, mits gameplaygedrag gelijk blijft en fouten zichtbaar blijven.

Audio Feedback

Sprong-audio moet het type sprong duidelijk maken.

Waarom:

- Een high jump moet niet alleen visueel waarneembaar zijn, maar zowel visueel als auditief.
- Het `weeh` -geluid past bij dat high-jump moment en maakt het effect direct herkenbaar.
- Dit ondersteunt timingleren, vooral bij snelle obstacle-combinaties.
- Normale sprong: gebruik `jump.wav` voor alle karakters.
- Versterkte/high jump (duck-jump, high-jump powerup of actieve jump shoes): gebruik `weeh.wav` voor alle karakters.

Shop En Powerups

De badger shop verschijnt in het menu en vlak voor boss fights.

Beschikbare powerups

- `Extra Life` :
  - icoon: hartje;
  - effect: absorbeert één fatale hit;
  - gebruik: blijft als voorraad bewaard tot een botsing het item verbruikt.
- `Shield` :
  - icoon: schild;
  - effect: tijdelijke bescherming;
  - duur: `SHOP_SHIELD_MS = 5000` .
- `Coin Boost` :
  - icoon: muntjes `x2` ;
  - effect: verdubbelt verzamelde coins tijdelijk;
  - duur: `SHOP_COIN_BOOST_MS = 60000` .
- `Jump Shoes` :
  - icoon: schoenen;
  - effect: hogere sprongen tijdelijk;
  - duur: `SHOP_JUMP_SHOES_MS = 30000` .

Activatiegedrag

- Aankopen in het startmenu worden bij run-start geactiveerd.

- Aankopen in de pre-boss shop worden geactiveerd zodra de speler de shop-overlay afsluit.
- Het schild en extra leven zijn verdedigende lagen; coin boost en jump shoes zijn tijdelijke boosts.

UI-richting

- De vier shop-items horen icon-first te zijn.
- Dezelfde iconen mogen later ook in de HUD gebruikt worden zodra losse uitgesneden assets beschikbaar zijn.
- De huidige HUD mag daarom iconen naast tekst tonen in plaats van alleen tekstregels.

Boss Entrances

Voor elk boss level stopt de endless-runner-flow kort in een statische hubscene.

- De speler kan daar links/rechts bewegen zoals in een boss arena.
- Links staat de badger shop.
- Rechts staat de entrance naar het gevecht.
- Interactie gebeurt met `Pijl omlaag` .

Mini bosses

- Voor level 4 en level 7 staat rechts een arena-ingang.
- De speler kan eerst shoppen en daarna bewust het gevecht starten.
- Level 6 heeft een afwijkende minibossflow: aan het einde van de pipe/flappy flight-sectie verschijnt een Zeppelin in de lucht in plaats van een ground-hub.
- Die Zeppelin wordt verslagen vanuit het eigen gele vliegtuig met een schot-aanval; de boss is dus onderdeel van `flight_mode` en niet van de gewone boss hub.

Zeppelin miniboss gedrag

- De Zeppelin komt eerst de stad in tijdens een korte `approach` -fase; daarna start pas de echte fight-fase.
- Tijdens deze encounter blijft de speler in `flight_mode` ; er is dus geen ground reset of gewone boss hub tussen intro en gevecht.
- De Zeppelin gebruikt een sprite-based render zodra `assets/zeppelin.png` beschikbaar is; alleen zonder asset blijft de oudere procedurale fallback actief.
- Tijdens het gevecht kan het vliegtuig `3` treffers van zeppelin-projectiles opvangen voordat het neerstort.
- Na de eerste treffer rookt het vliegtuig periodiek ongeveer elke `4` seconden.

- Na de tweede treffer rookt het vliegtuig periodiek ongeveer elke `2` seconden.
- De derde treffer door een zeppelin-projectile veroorzaakt een crash.
- Een botsing met een pipe blijft direct fataal; pipe-collisions gebruiken dus geen hitpoint-systeem.
- De post-boss overgang mag de luchtarena pas loslaten nadat de defeat/explosion sequence visueel klaar is.
- Daarna moet de speler logisch de gewone wereld in vallen vanuit de actuele hoogte van het gevecht, niet via een kunstmatige teleport naar een vaste y-positie.

Waarom:

- De overgang van level 5 naar 6 bouwt al een luchtmechanic op; een luchtboss benut die bestaande skill direct.
- Een ground-hub zou de opgebouwde flight-spanning onnodig onderbreken.
- De speler moet dezelfde besturing en voertuigfantasie behouden tijdens het gevecht.
- Een 3-hit vliegtuigstate maakt de fight minder binair en leesbaarder zonder pipes of arena-positioning ongevaarlijk te maken.
- Rookfeedback maakt schade zichtbaar in de wereld zelf, in lijn met het principe dat gameplay-impact niet alleen in tekst mag zitten.

Eindbaas level 10

- Voor de coyote-boss staat rechts een pijp.
- `Pijl omlaag` op de pijp start de ondergrondse boss arena.
- De arena gebruikt een donkerdere, grijze grot-achtige achtergrond.
- Grote vallende bommen tonen een gele landingsgloed zolang ze nog in de lucht zijn.
- Zodra een grote bom ontploft, licht de cave kort op.

Documentatiegraad bosses

- De globale boss-entrances en de specifieke level-6 en level-10 regels zijn nu beschreven in deze SGB.
- Het volledige encountergedrag van de bird boss, cactus boss en final boss is nog niet overal systematisch uitgeschreven.
- Verdere iteraties op boss-balans, arena-flow en visuele feedback horen die encounterregels later ook expliciet per boss aan te vullen.

Level Systeem

- De game heeft `10` levels.

- Hoofdstuk `1` duurt `6` obstacles.
- Elk volgend level vraagt `3` obstacles meer dan het vorige.
- Bij elk nieuw level stijgt de scrollsnelheid met factor `1.1`.
- Score en level-progressie zijn losgekoppeld:
  - score komt uit punten, coins en boss rewards;
  - level-progressie komt uit het aantal cleared obstacles.
- Boss rewards volgen de benodigde hits:
  - miniboss `1`: `15` punten;
  - miniboss `flight Zeppelin`: `18` punten;
  - miniboss `2`: `15` punten;
  - eindbaas: `35` punten.

Obstakelgeneratiebeleid

- Levels `1` t/m `7` zijn primair leerbaar en gebruiken vaste obstaclepatronen.
- Vanaf level `8` is beperkte variatie toegestaan, maar alleen met curated templates (geen vrije RNG-combinatie van losse obstakels).
- Elke template voor level `8+` moet handmatig speelbaar gevalideerd zijn.
- Validatieregel: ook als een high-jump powerup leeg is, moet de speler met normale sprong een uitwijkroute of haalbare timing houden.
- Templates die directe "soft-lock" situaties kunnen veroorzaken (zoals te krappe multi-cactusketens zonder herstelmoment) zijn niet toegestaan.

Aantal obstacles per level

- Level `1`: `6` obstacles.
- Level `2`: `9` obstacles (`15` totaal).
- Level `3`: `12` obstacles (`27` totaal).
- Level `4`: `15` obstacles (`42` totaal).
- Level `5`: `18` obstacles (`60` totaal).
- Level `6`: `21` obstacles (`81` totaal).
- Level `7`: `24` obstacles (`105` totaal).
- Level `8`: `27` obstacles (`132` totaal).
- Level `9`: `30` obstacles (`162` totaal).
- Level `10`: `33` obstacles (`195` totaal).

Levels

Level 1: `Enter Cactus Land...`

- Introductie van de woestijn.
- Basisobstakels: lage cactus, hoge cactus en lage vogel.
- Nog geen slang.

Level 2: `Snake Sands`

- Meer druk in de woestijn.
- Slangen komen erbij.
- Timing tussen vogel, cactus en slang varieert meer.

Level 3: `High Jump Ridge`

- Verticale sprongen worden belangrijker.
- Torencactus en high-jumpflow krijgen nadruk.
- Jump blocks kunnen regen, natte grond en bloemen triggeren.

Level 4: `Bird Boss Canyon`

- Eerste minibossfase.
- Reuzenvogel-boss.
- Shop-hub vóór de arena-ingang.

Level 5: `Fly away`

- Eerste vliegtuighoofdstuk.
- Pijpen verschijnen als obstakels.
- Vliegtuig kan als pickup flight mode starten.

Level 6: `Storm Track`

- Tweede vliegtuighoofdstuk.
- Flight mode loopt door over level 5 en 6.
- Pijpen blijven actief als obstakeltype.
- Aan het eind van deze luchtsectie verschijnt de Zeppelin-tussenbaas.
- De speler blijft in het gele vliegtuig en schiet de Zeppelin neer vanuit de lucht.

Level 7: `Cactus Fortress`

- Tweede minibossfase.
- Grondgevecht tegen de reuzencactus.
- Shop-hub vóór de arena-ingang.

Level 8: `Wild Flats`

- Meer tempo en krappe obstacle-combinaties.
- Multi-cactus packs vragen snelle landingen.

Level 9: `Last Stretch`

- Voorbereiding op de eindbaas.
- Hogere reactiedruk en minder hersteltijd.

Level 10: `Giant Town`

- Eindbaasfase.
- Reuzenvariant per karakter:
  - dino: `ReuzenDino` ;
  - cowboy: `ReuzenCowboy` ;
  - roadrunner: `ReuzenCoyote` .
- Voor de coyote-route loopt de speler eerst via de shop-hub naar een pijp.
- De coyote wordt verslagen door `5` grote bommen terug te gooien.

Referenties

- Speluitleg en projectnotities: dino.md
- Contributierichtlijnen: CONTRIBUTING.md
- Agent/projectrichtlijnen: AGENTS.md

Audioformaat en webcompatibiliteit

Voor audio-assets in web builds (pygbag) geldt:

> ".wav and .mp3 are safe, .ogg is not always supported on all browsers"
> (pygbag README, 2024, https://github.com/pygame-web/pygbag#assets)

- .wav en .mp3 werken in alle moderne browsers, inclusief Safari/iOS.
- .ogg werkt niet in Safari/iOS en is dus niet universeel web-compatibel.
- Zie ook: MDN Web Docs: Audio codecs

Waarom

- Fallbacks op .ogg/.mp3/.wav maken foutmeldingen onduidelijk en maskeren assetproblemen.
- Alleen het juiste, verwachte bestand wordt geladen; ontbreekt dat, dan volgt een duidelijke foutmelding.
- Dit voorkomt verwarring over ontbrekende of niet-universeel ondersteunde audioformaten en zorgt voor dev/prod parity.